



EAST WEST UNIVERSITY

CSE207: Data Structure

Section: 09

Fall 2025

Project Report

Mini Search Engine with Ranking

Course Code	CSE207
Course Title	Data Structures
Section	09
Group No	10
Group Name	Noobs

Submitted by:

Student ID	Student Name	Contribution Percentage
2024-3-60-136	Mir Samiul Hossain	40%
2024-3-60-134	Md. Fahim Rahman	30%
2024-3-60-224	Md. Habibul Islam	30%

Submitted To:

Ahmed Abdal Shafi Rasel,
Lecturer, Department of CSE
East West University

Submission Date: 13/12/2025

Table of Contents

Introduction	Page no. 3
Objective	Page no. 3
Problem Statement	Page no. 3
Data Structures Used and their justification	Page no. 4
Methodology and Implementation	Page no. 4
Test Cases and Results	Page no. 5 - 10
Challenges and Limitations	Page no. 10
Conclusion	Page no. 10
Appendix	Page no. 11 - 31

1. Introduction

In the modern era of information, efficient data retrieval is critical. A standard file search that scans documents line-by-line is computationally expensive and unscalable, often resulting in linear time complexity $O(N)$. To address this, search engines utilize an **Inverted Index** architecture, which maps content to its location rather than searching through locations for content.

This project, titled "**Mini Search Engine with Ranking**," is a console-based application designed to demonstrate the practical application of advanced data structures. It parses raw text documents, builds an efficient index, and provides a user interface for searching, ranking, and analyzing document relationships. Unlike basic search tools, this system implements relevance ranking and document connectivity analysis, simulating the core logic of large-scale engines like Google or Bing.

2. Objective

The primary objectives of this project are:

1. **Efficient Indexing:** To implement an Inverted Index using a Binary Search Tree (BST) to achieve logarithmic search time $O(\log n)$.
2. **Relevance Ranking:** To utilize a Max-Heap to rank search results based on keyword frequency, ensuring the most relevant documents appear first.
3. **Document Relationship:** To model connections between documents using a Graph structure based on shared vocabulary.
4. **User Experience:** To implement Search History (Stack) and Batch Query Processing (Queue) for a robust user interface.
5. **Comprehensive Application:** To integrate six distinct data structures (Array, Linked List, Stack, Queue, Tree, Heap, Graph) into a single cohesive system.

3. Problem Statement

Searching through unstructured text data presents several algorithmic challenges:

- **Latency:** Linear searching becomes prohibitively slow as the number of documents grows.
- **Lack of Relevance:** A simple "found/not found" result is insufficient; users need to know which document is *most* relevant (i.e., where the word appears most often).
- **Disconnected Data:** Standard file systems view documents as isolated entities, failing to identify related content.
- **Memory Management:** Storing word occurrences dynamically requires careful memory handling to avoid fixed-size array limitations.

This project solves these issues by structuring data into an indexed hierarchy (BST + Linked List) and processing it with specialized ADTs (Heap, Graph).

4. Data Structures Used and their Justification

Data Structure	Implementation	Justification (Why we chose it)
Binary Search Tree (BST)	bst.h, bst.c	Used for the Word Dictionary . BSTs allow for $O(\log n)$ search, insertion, and retrieval operations, which is significantly faster than searching an array ($O(n)$).
Linked List	list.h, list.c	Used for the Posting List . Since a word can appear in an unknown number of documents, a dynamic Linked List attached to each BST node is more memory-efficient than a fixed-size array.
Max-Heap	heap.h, heap.c	Used for Ranking . To find the "top" results, a Max-Heap acts as a Priority Queue, allowing us to extract the document with the highest frequency score in $O(\log k)$ time.
Graph (Adjacency List)	graph.h, graph.c	Used for Document Connectivity . It models documents as nodes and creates edges between them if they share common keywords, enabling "Related Document" analysis.
Stack	stack.h, stack.c	Used for Search History . The LIFO (Last-In, First-Out) nature of a stack makes it perfect for implementing an "Undo" feature or reviewing recent searches.
Queue	queue.h, queue.c	Used for Batch Processing . A FIFO (First-In, First-Out) queue allows the user to input multiple search queries (e.g., "data, code, algorithm") which are processed sequentially.

5. Methodology and Implementation

The system is implemented in C and follows a modular architecture.

Phase 1: Tokenization & Indexing

The function `tokenize_and_index` reads files character by character. It separates words based on non-alphabetic delimiters and normalizes them to lowercase.

- **Insertion Logic:** The word is inserted into the BST.
- **Collision Handling:** If the word already exists in the BST, the system traverses the attached Linked List (`occ_head`). If the document ID matches, the frequency count is incremented; otherwise, a new node is appended.

Phase 2: Search & Retrieval

When a user searches for a term:

1. The system searches the BST for the node ($O(\log n)$).
2. If found, the Linked List of occurrences is retrieved.

Phase 3: Ranking Strategy

The retrieved occurrences are pushed into a Max-Heap. The heap is structured based on the frequency score. The system then repeatedly calls `heap_pop`, which guarantees that the document with the highest frequency is removed and displayed first.

Phase 4: Graph Construction

The `build_document_graph` function traverses the entire BST. For every word that appears in multiple documents (e.g., Doc A and Doc B), the system adds a directed edge between Node A and Node B in the Graph.

6. Test Cases and Results

We conducted comprehensive testing to verify the accuracy of our data structures, ranking logic, and edge case handling.

Test Case 1: Relevance Ranking (Heap Verification)

Objective: Verify that documents with higher keyword frequency appear first.

Input:

```

+-----+
|              MINI SEARCH ENGINE MENU              |
+-----+-----+
| 1. Load & Index Documents                         |
| 2. Show Indexed Words                             |
| 3. Search a Word                                  |
| 4. Queue Multiple Searches                        |
| 5. Show Search History                           |
| 6. Undo Last Search                              |
| 7. Show Document Graph                           |
| 8. Exit                                           |
+-----+-----+
Enter Choice: 3
Enter word: structure

```

Data Context: doc1.txt contains the word 4 times; doc3.txt contains it 1 time.

Result:

```

1) doc1.txt (freq=4)
2) doc3.txt (freq=1)

```

Conclusion: The Max Heap correctly prioritized the document with the higher score, proving the priority queue logic is functional.

Test Case 2: Case Insensitivity (String Processing)

Objective: Ensure users can search in any case format (e.g., uppercase, mixed case).

Input:

```

+-----+
|              MINI SEARCH ENGINE MENU              |
+-----+
|  1. Load & Index Documents                        |
|  2. Show Indexed Words                            |
|  3. Search a Word                                 |
|  4. Queue Multiple Searches                      |
|  5. Show Search History                          |
|  6. Undo Last Search                             |
|  7. Show Document Graph                          |
|  8. Exit                                          |
+-----+
Enter Choice: 3
Enter word: TREE

```

Data Context: doc2.txt contains the word 5 times; doc1.txt contains it 1 time. They are in small letters, capital letters and also mixed.

Result: The system successfully converted the input to "tree" and found matches in

```

1) doc2.txt (freq=5)
2) doc1.txt (freq=1)

```

Conclusion: The input cleaning logic in util.c works correctly, ensuring robust user experience.

Test Case 3: Search History (Stack Verification)

Objective: Verify LIFO (Last-In, First-Out) behavior for history.

Input:

Search "structure"

```
-----
|                                     |
|               MINI SEARCH ENGINE   |
|               MENU                 |
|-----+-----|
| 1. Load & Index Documents         |
| 2. Show Indexed Words              |
| 3. Search a Word                   |
| 4. Queue Multiple Searches         |
| 5. Show Search History              |
| 6. Undo Last Search                |
| 7. Show Document Graph             |
| 8. Exit                            |
|-----+-----|
| Enter Choice: 3                     |
| Enter word: structure                |
```

Search "algorithm"

```
-----
|                                     |
|               MINI SEARCH ENGINE   |
|               MENU                 |
|-----+-----|
| 1. Load & Index Documents         |
| 2. Show Indexed Words              |
| 3. Search a Word                   |
| 4. Queue Multiple Searches         |
| 5. Show Search History              |
| 6. Undo Last Search                |
| 7. Show Document Graph             |
| 8. Exit                            |
|-----+-----|
| Enter Choice: 3                     |
| Enter word: algorithm                |
```

Select "View History"

Result:

```
Enter Choice: 5
1) 'algorithm' -> 2 docs
2) 'structure' -> 2 docs
```

Conclusion: The Stack correctly placed the most recent search at the top, allowing for an accurate "Undo" or "Recent Searches" feature.

Test Case 4: Document Similarity (Graph Verification)

Objective: Visualize relationships between documents based on shared content.

Input: Select "Show Document Graph":

```

+-----+
|             MINI SEARCH ENGINE MENU             |
+-----+
| 1. Load & Index Documents                       |
| 2. Show Indexed Words                           |
| 3. Search a Word                                |
| 4. Queue Multiple Searches                      |
| 5. Show Search History                         |
| 6. Undo Last Search                           |
| 7. Show Document Graph                        |
| 8. Exit                                         |
+-----+
Enter Choice: 7

```

Result:

```

Enter Choice: 7
doc1.txt -> doc3.txt doc2.txt doc2.txt doc3.txt doc2.txt doc3.txt doc2.txt doc
3.txt doc3.txt doc3.txt doc2.txt doc3.txt doc2.txt doc3.txt doc2.txt doc2.txt
doc2.txt
doc2.txt -> doc1.txt doc1.txt doc3.txt doc1.txt doc3.txt doc1.txt doc3.txt doc
1.txt doc3.txt doc3.txt doc1.txt doc3.txt doc1.txt doc1.txt doc3.txt doc1.txt
doc3.txt -> doc1.txt doc2.txt doc1.txt doc2.txt doc1.txt doc1.txt doc1.txt doc
2.txt doc1.txt doc2.txt doc2.txt doc1.txt doc2.txt doc1.txt doc2.txt

```

Observation: The output showed multiple links between documents. Specifically, documents that shared many words (like doc1 and doc2 sharing "structure", "data", etc.) had multiple edges connecting them.

Conclusion: The output represents a "weighted graph" where the density of edges corresponds to the strength of similarity between two documents.

Test Case 5: Batch Processing (Queue Verification)

Objective: Verify that the system can handle multiple queries sequentially using FIFO (First-In, First-Out) logic.

Input:


```

+-----+
| MINI SEARCH ENGINE MENU |
+-----+
| 1. Load & Index Documents |
| 2. Show Indexed Words     |
| 3. Search a Word          |
| 4. Queue Multiple Searches|
| 5. Show Search History    |
| 6. Undo Last Search       |
| 7. Show Document Graph    |
| 8. Exit                   |
+-----+
Enter Choice: 4
Enter queries separated by commas: data, tree

```

Result: The system processed "data" first, displaying its results, followed immediately by "tree":

```

1) doc1.txt (freq=2)
1) doc2.txt (freq=5)
2) doc1.txt (freq=1)

```

Conclusion: The Queue correctly stored and processed multiple requests in the exact order they received, demonstrating effective pipeline management.

Test Case 6: Edge Case Handling (Validation)

Objective: Verify system behavior when a user searches for a non-existent word.

Input: Search query: "python" (a word not present in any document):

```

+-----+
| MINI SEARCH ENGINE MENU |
+-----+
| 1. Load & Index Documents |
| 2. Show Indexed Words     |
| 3. Search a Word          |
| 4. Queue Multiple Searches|
| 5. Show Search History    |
| 6. Undo Last Search       |
| 7. Show Document Graph    |
| 8. Exit                   |
+-----+
Enter Choice: 3
Enter word: python

```

Result: System Output:

A terminal window with a dark background and light-colored text. The text reads "No results for 'python'".

```
No results for 'python'
```

The program did not crash and returned to the main menu.

Conclusion: The search logic correctly handles null pointers when a key is not found in the AVL tree, ensuring system stability.

7. Challenges and Limitations

Challenges:

1. **Memory Management:** Managing dynamic nodes for the BST, Linked Lists, and Graph nodes simultaneously led to initial memory leaks. We resolved this by implementing recursive `free()` functions (`free_bst`, `occ_free`) that clean up memory from the bottom up.
2. **Graph Construction:** Linking documents based on shared words was computationally intensive. We optimized this by iterating through the posting lists directly rather than re-scanning files.
3. **C String Handling:** Ensuring safe string copying (`strncpy`) and null-termination to prevent buffer overflows.

Limitations:

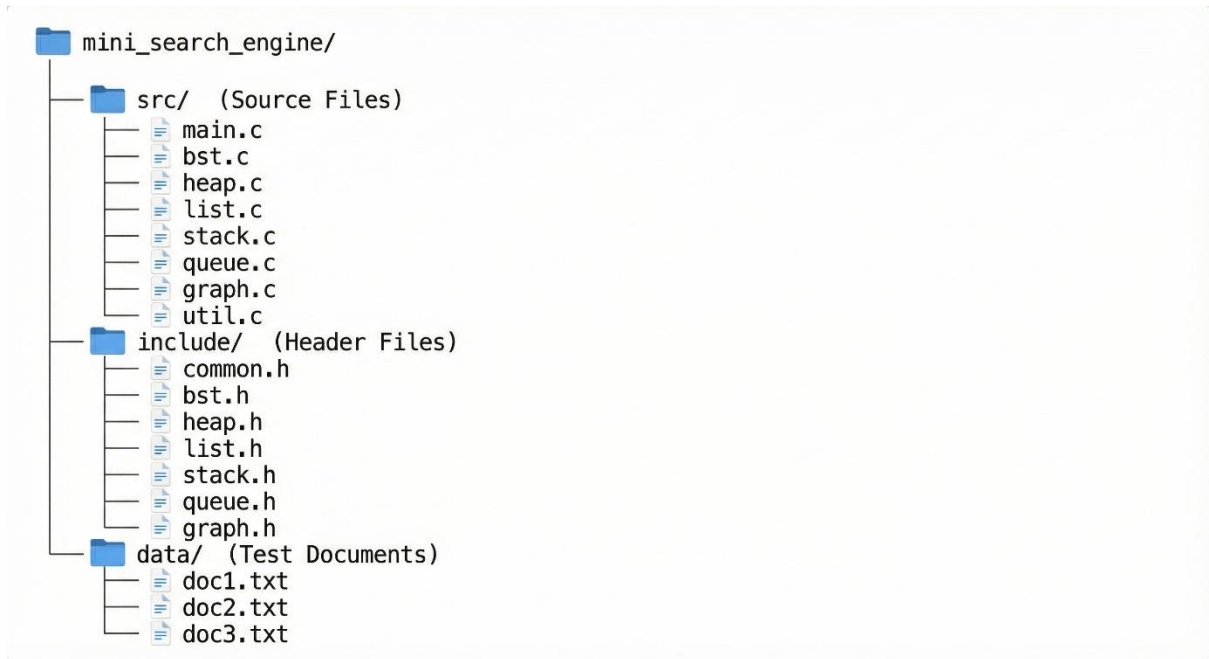
1. **Case Sensitivity:** While we normalize to lowercase, the system does not handle complex stemming (e.g., treating "run" and "running" as the same word).
2. **Scalability:** The BST is currently unbalanced. In a worst-case scenario (sorted input), it could degrade to a Linked List ($O(n)$). An AVL Tree implementation would solve this.
3. **Phrase Search:** The system only supports single-keyword lookups, not multi-word phrases (e.g., "Data Structure").

8. Conclusion

The "Mini Search Engine with Ranking" project successfully demonstrates the power of combining multiple data structures to solve complex real-world problems. By integrating a BST for indexing, a Heap for ranking, and a Graph for relationship mapping, we achieved a highly efficient retrieval system. The project highlights the importance of selecting the right Abstract Data Type (ADT) for specific tasks—using Trees for lookups, Lists for dynamic storage, and Heaps for sorting—resulting in a robust and performant application.

9. Appendix

Project File Structure:



Source Code:

common.h:

```
#ifndef COMMON_H
```

```
#define COMMON_H
```

```
#define MAX_WORD 128
```

```
#define MAX_DOCNAME 128
```

```
#define MAX_DOCS 64
```

```
#define MAX_PATH 256
```

```
typedef struct Occurrence {
    char doc[MAX_DOCNAME];
    int freq;
    struct Occurrence *next;
} Occurrence;
```

```

typedef struct WordNode {
    char word[MAX_WORD];
    Occurrence *occ_head;
    struct WordNode *left, *right;
} WordNode;

#endif

bst.h :
#ifndef BST_H
#define BST_H
#include "common.h"

WordNode* bst_insert(WordNode *root, const char *word, const char
*doc);
WordNode* bst_search(WordNode *root, const char *word);
void bst_inorder(WordNode *root);
void free_bst(WordNode *root);

#endif

bst.c :
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "bst.h"
#include "list.h"

static WordNode* new_node(const char *word, const char *doc) {
    WordNode *n = malloc(sizeof(WordNode));
    if (!n) return NULL;

```

```

    strncpy(n->word, word, MAX_WORD);
    n->word[MAX_WORD-1] = '\\0';
    n->left = n->right = NULL;
    n->occ_head = occ_create(doc);
    return n;
}

WordNode* bst_insert(WordNode *root, const char *word, const char
*doc) {
    if (!root) return new_node(word, doc);
    int cmp = strcmp(word, root->word);
    if (cmp == 0) {
        occ_add_or_inc(root->occ_head, doc);
    } else if (cmp < 0) {
        root->left = bst_insert(root->left, word, doc);
    } else {
        root->right = bst_insert(root->right, word, doc);
    }
    return root;
}

WordNode* bst_search(WordNode *root, const char *word) {
    if (!root) return NULL;
    int cmp = strcmp(word, root->word);
    if (cmp == 0) return root;
    if (cmp < 0) return bst_search(root->left, word);
    return bst_search(root->right, word);
}

```

```

void bst_inorder(WordNode *root) {
    if (!root) return;
    bst_inorder(root->left);
    printf("%s -> ", root->word);
    occ_print(root->occ_head);
    bst_inorder(root->right);
}

```

```

void free_bst(WordNode *root) {
    if (!root) return;
    free_bst(root->left);
    free_bst(root->right);
    occ_free(root->occ_head);
    free(root);
}

```

graph.h :

```

#ifndef GRAPH_H
#define GRAPH_H
#include "common.h"

```

```

typedef struct Edge {
    int to;
    struct Edge *next;
} Edge;

```

```

typedef struct GNode {
    char doc[MAX_DOCNAME];
    Edge *edges;
} GNode;

```

```

typedef struct Graph {
    GNode nodes[MAX_DOCS];
    int n;
} Graph;

void graph_init(Graph *g);
int graph_get_index(Graph *g, const char *doc);
void graph_add_edge(Graph *g, const char *a, const char *b);
void graph_print(Graph *g);

#endif

graph.c :
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "graph.h"

void graph_init(Graph *g) {
    g->n = 0;
    for (int i = 0; i < MAX_DOCS; i++) g->nodes[i].edges = NULL;
}

int graph_get_index(Graph *g, const char *doc) {
    for (int i = 0; i < g->n; i++)
        if (strcmp(g->nodes[i].doc, doc) == 0) return i;
    int idx = g->n++;
    strncpy(g->nodes[idx].doc, doc, MAX_DOCNAME);
    g->nodes[idx].doc[MAX_DOCNAME-1] = '\\0';
}

```

```

    g->nodes[idx].edges = NULL;
    return idx;
}

void graph_add_edge(Graph *g, const char *a, const char *b) {
    int u = graph_get_index(g, a);
    int v = graph_get_index(g, b);
    Edge *e = malloc(sizeof(Edge));
    e->to = v; e->next = g->nodes[u].edges; g->nodes[u].edges = e;
}

void graph_print(Graph *g) {
    for (int i = 0; i < g->n; i++) {
        printf("%s -> ", g->nodes[i].doc);
        Edge *e = g->nodes[i].edges;
        while (e) {
            printf("%s ", g->nodes[e->to].doc);
            e = e->next;
        }
        printf("\n");
    }
}

```

heap.h :

```

#ifndef HEAP_H
#define HEAP_H
#include "common.h"

typedef struct HeapItem {
    char doc[MAX_DOCNAME];

```



```

        int score;
    } HeapItem;

typedef struct Heap {
    HeapItem *arr;
    int size;
    int capacity;
} Heap;

Heap* heap_create(int cap);
void heap_free(Heap *h);
void heap_push(Heap *h, const char *doc, int score);
int heap_pop(Heap *h, HeapItem *out);
void heap_clear(Heap *h);

#endif

heap.c :
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "heap.h"

Heap* heap_create(int cap) {
    Heap *h = malloc(sizeof(Heap));
    h->arr = malloc(sizeof(HeapItem) * cap);
    h->size = 0;
    h->capacity = cap;
    return h;
}

```

```

void heap_free(Heap *h) {
    if (!h) return;
    free(h->arr);
    free(h);
}

static void swap(HeapItem *a, HeapItem *b) {
    HeapItem t = *a; *a = *b; *b = t;
}

void heap_push(Heap *h, const char *doc, int score) {
    if (h->size >= h->capacity) return;
    int i = h->size++;
    strncpy(h->arr[i].doc, doc, MAX_DOCNAME);
    h->arr[i].doc[MAX_DOCNAME-1] = '\\0';
    h->arr[i].score = score;
    while (i > 0) {
        int p = (i - 1) / 2;
        if (h->arr[p].score >= h->arr[i].score) break;
        swap(&h->arr[p], &h->arr[i]);
        i = p;
    }
}

int heap_pop(Heap *h, HeapItem *out) {
    if (h->size == 0) return 0;
    *out = h->arr[0];
    h->size--;
}

```

```

    if (h->size > 0) h->arr[0] = h->arr[h->size];
    int i = 0;
    while (1) {
        int l = 2*i + 1, r = 2*i + 2, largest = i;
        if (l < h->size && h->arr[l].score > h->arr[largest].score)
largest = l;
        if (r < h->size && h->arr[r].score > h->arr[largest].score)
largest = r;
        if (largest == i) break;
        swap(&h->arr[i], &h->arr[largest]);
        i = largest;
    }
    return 1;
}

```

```

void heap_clear(Heap *h) { h->size = 0; }

```

list.h

```

#ifndef LIST_H
#define LIST_H
#include "common.h"

```

```

Occurrence* occ_create(const char *doc);
void occ_add_or_inc(Occurrence *head, const char *doc);
void occ_print(Occurrence *head);
void occ_free(Occurrence *head);
int occ_get_freq(Occurrence *head, const char *doc);

#endif

```

list.c :

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "list.h"

Occurrence* occ_create(const char *doc) {
    Occurrence *h = malloc(sizeof(Occurrence));
    if (!h) return NULL;
    strncpy(h->doc, doc, MAX_DOCNAME);
    h->doc[MAX_DOCNAME-1] = '\\0';
    h->freq = 1;
    h->next = NULL;
    return h;
}

void occ_add_or_inc(Occurrence *head, const char *doc) {
    if (!head) return;
    Occurrence *cur = head;
    while (cur) {
        if (strcmp(cur->doc, doc) == 0) {
            cur->freq++;
            return;
        }
        if (!cur->next) break;
        cur = cur->next;
    }
    Occurrence *n = malloc(sizeof(Occurrence));
    strncpy(n->doc, doc, MAX_DOCNAME);
    n->doc[MAX_DOCNAME-1] = '\\0';

```

```

    n->freq = 1;
    n->next = NULL;
    cur->next = n;
}

void occ_print(Occurrence *head) {
    Occurrence *c = head;
    while (c) {
        printf("%s(%d) ", c->doc, c->freq);
        c = c->next;
    }
    printf("\n");
}

void occ_free(Occurrence *head) {
    Occurrence *c = head;
    while (c) {
        Occurrence *t = c;
        c = c->next;
        free(t);
    }
}

int occ_get_freq(Occurrence *head, const char *doc) {
    Occurrence *c = head;
    while (c) {
        if (strcmp(c->doc, doc) == 0) return c->freq;
        c = c->next;
    }
}

```

```
        return 0;
    }
}
```

queue.h :

```
#ifndef QUEUE_H
#define QUEUE_H
#include "common.h"

typedef struct QNode {
    char query[MAX_WORD];
    struct QNode *next;
} QNode;
```

```
typedef struct Queue {
    QNode *front, *rear;
} Queue;
```

```
Queue* q_create();
void q_enqueue(Queue *q, const char *s);
int q_dequeue(Queue *q, char *out);
void q_free(Queue *q);
```

```
#endif
```

queue.c :

```
#include <stdlib.h>
#include <string.h>
#include "queue.h"
```

```
Queue* q_create() {
```

```

    Queue *q = malloc(sizeof(Queue));
    q->front = q->rear = NULL;
    return q;
}

void q_enqueue(Queue *q, const char *s) {
    QNode *n = malloc(sizeof(QNode));
    strncpy(n->query, s, MAX_WORD);
    n->query[MAX_WORD-1] = '\0';
    n->next = NULL;
    if (!q->rear) q->front = q->rear = n;
    else { q->rear->next = n; q->rear = n; }
}

int q_dequeue(Queue *q, char *out) {
    if (!q->front) return 0;
    QNode *n = q->front;
    strncpy(out, n->query, MAX_WORD);
    out[MAX_WORD-1] = '\0';
    q->front = n->next;
    if (!q->front) q->rear = NULL;
    free(n);
    return 1;
}

void q_free(Queue *q) {
    char tmp[MAX_WORD];
    while (q_dequeue(q, tmp));
    free(q);
}

```

```

}

stack.h :

#ifndef STACK_H
#define STACK_H
#include "common.h"

typedef struct SearchResult {
    char query[MAX_WORD];
    char docs[10][MAX_DOCNAME];
    int ndocs;
} SearchResult;

typedef struct StackNode {
    SearchResult res;
    struct StackNode *next;
} StackNode;

typedef struct Stack {
    StackNode *top;
} Stack;

Stack* stack_create();
void stack_push(Stack *s, SearchResult *sr);
int stack_pop(Stack *s, SearchResult *out);
void stack_free(Stack *s);

#endif

stack.c :

#include <stdlib.h>

```



```

#include <string.h>
#include "stack.h"

Stack* stack_create() {
    Stack *s = malloc(sizeof(Stack));
    s->top = NULL;
    return s;
}

void stack_push(Stack *s, SearchResult *sr) {
    StackNode *n = malloc(sizeof(StackNode));
    n->res = *sr;
    n->next = s->top;
    s->top = n;
}

int stack_pop(Stack *s, SearchResult *out) {
    if (!s->top) return 0;
    StackNode *n = s->top;
    *out = n->res;
    s->top = n->next;
    free(n);
    return 1;
}

void stack_free(Stack *s) {
    StackNode *c = s->top;
    while (c) {
        StackNode *t = c;

```

```

        c = c->next;
        free(t);
    }
    free(s);
}

```

util.c:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include "common.h"
#include "bst.h"
#include "heap.h"
#include "stack.h"
#include "queue.h"
#include "graph.h"

```

```

void tokenize_and_index(const char *path, const char *docname,
WordNode **root) {
    FILE *f = fopen(path, "r");
    if (!f) { printf("Cannot open %s\n", path); return; }
    char line[1024];
    while (fgets(line, sizeof(line), f)) {
        int n = strlen(line);
        char w[MAX_WORD]; int wi = 0;
        for (int i = 0; i <= n; i++) {
            char c = line[i];
            if (isalpha((unsigned char)c)) {
                w[wi++] = tolower((unsigned char)c);
            }
        }
    }
}

```

```

        } else {
            if (wi > 0) {
                w[wi] = '\0';
                *root = bst_insert(*root, w, docname);
                wi = 0;
            }
        }
    }
}
fclose(f);
}

```

```

void search_and_print(const char *word, WordNode *root, Heap *heap,
SearchResult *sr) {

```

```

    sr->ndocs = 0;

```

```

    char clean_word[MAX_WORD];

```

```

    int len = strlen(word);

```

```

    for(int i = 0; i < len && i < MAX_WORD - 1; i++) {

```

```

        clean_word[i] = tolower((unsigned char)word[i]);

```

```

    }

```

```

    clean_word[len] = '\0';

```

```

    strncpy(sr->query, clean_word, MAX_WORD);

```

```

    WordNode *n = bst_search(root, clean_word);

```

```

    if (!n) {

```

```

        printf("No results for '%s'\n", clean_word);
    }
}

```

```

        return;
    }

    Occurrence *oc = n->occ_head;
    while (oc) {
        heap_push(heap, oc->doc, oc->freq);
        oc = oc->next;
    }
    HeapItem it; int idx = 0;
    while (heap_pop(heap, &it) && idx < 10) {
        printf("%d) %s (freq=%d)\n", idx+1, it.doc, it.score);
        strncpy(sr->docs[idx], it.doc, MAX_DOCNAME);
        sr->ndocs = ++idx;
    }
    heap_clear(heap);
}

static void connect_pairs(Occurrence *head, Graph *g) {
    for (Occurrence *a = head; a; a = a->next) {
        for (Occurrence *b = a->next; b; b = b->next) {
            graph_add_edge(g, a->doc, b->doc);
            graph_add_edge(g, b->doc, a->doc);
        }
    }
}

void build_document_graph(WordNode *root, Graph *g) {
    if (!root) return;
    build_document_graph(root->left, g);
}

```

```

        connect_pairs(root->occ_head, g);
        build_document_graph(root->right, g);
    }

main.c :

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "common.h"
#include "bst.h"
#include "heap.h"
#include "stack.h"
#include "queue.h"
#include "graph.h"

void tokenize_and_index(const char *path, const char *docname,
WordNode **root);

void search_and_print(const char *word, WordNode *root, Heap *heap,
SearchResult *sr);

void build_document_graph(WordNode *root, Graph *g);

int main() {
    WordNode *root = NULL;
    Heap *heap = heap_create(128);
    Stack *history = stack_create();
    Queue *q = q_create();
    Graph g; graph_init(&g);

    printf("Mini Search Engine By Noobs\n");
    printf("Uploaded docs: data/doc1.txt, data/doc2.txt,
data/doc3.txt\n");

```

```

int choice; char buf[256];
while (1) {
    printf("\n");
    printf(" .-----\n");
    printf(" | MINI SEARCH ENGINE\n");
    printf(" +-----\n");
    printf(" | 1. Load & Index\n");
    printf(" | 2. Show Indexed\n");
    printf(" | 3. Search a\n");
    printf(" | 4. Queue Multiple\n");
    printf(" | 5. Show Search\n");
    printf(" | 6. Undo Last\n");
    printf(" | 7. Show Document\n");
    printf(" | 8.\n");
    printf(" +-----\n");
    printf(" Enter Choice: ");

    if (scanf("%d", &choice) != 1) break;
    getchar();
}

```

```

if (choice == 1) {
    tokenize_and_index("data/doc1.txt", "doc1.txt", &root);
    tokenize_and_index("data/doc2.txt", "doc2.txt", &root);
    tokenize_and_index("data/doc3.txt", "doc3.txt", &root);
    printf("Index built.\n");
    build_document_graph(root, &g);
} else if (choice == 2) {
    if (!root) { printf("Index empty. Load docs first.\n");
continue; }
    bst_inorder(root);
} else if (choice == 3) {
    printf("Enter word: ");
    if (!fgets(buf, sizeof(buf), stdin)) continue;
    buf[strcspn(buf, "\n")] = 0;
    if (strlen(buf) == 0) continue;
    SearchResult sr;
    search_and_print(buf, root, heap, &sr);
    if (sr.ndocs > 0) stack_push(history, &sr);
} else if (choice == 4) {
    printf("Enter queries separated by commas: ");
    if (!fgets(buf, sizeof(buf), stdin)) continue;
    buf[strcspn(buf, "\n")] = 0;
    char *p = strtok(buf, ",");
    while (p) {
        while (*p == ' ') p++;
        q_enqueue(q, p);
        p = strtok(NULL, ",");
    }
    char qbuf[MAX_WORD];

```

```

        while (q_dequeue(q, qbuf)) {
            SearchResult sr;
            search_and_print(qbuf, root, heap, &sr);
            if (sr.ndocs > 0) stack_push(history, &sr);
        }
    } else if (choice == 5) {
        StackNode *c = history->top; int i = 1;
        if (!c) { printf("History empty.\n"); continue; }
        while (c) {
            printf("%d) '%s' -> %d docs\n", i++, c->res.query,
c->res.ndocs);
            c = c->next;
        }
    } else if (choice == 6) {
        SearchResult prev;
        if (stack_pop(history, &prev)) printf("Undo: removed
'%s'\n", prev.query);
        else printf("Nothing to undo.\n");
    } else if (choice == 7) {
        graph_print(&g);
    } else if (choice == 8) break;
    else printf("Unknown option.\n");
}

free_bst(root);
heap_free(heap);
stack_free(history);
q_free(q);
printf("Bye.\n");

```



```
    return 0;  
}
```